



دانشگاه تهران

پردیس دانشکده های فنی

دانشکده مهندسی برق و کامپیوتر

گزارش تمرین شماره سه

عنوان درس: مبانی رایانش توزیع شده

امین آقاکثیری – 810101381

مهدی نائینی – 810101536

بهار 1405

بخش اول: سوالات تئوری

چرا داده ها در سیستم های توزیع شده replicate می شوند؟

داده ها در سیستم های توزیع شده replicate می شوند چون این کار چند هدف مهم را هم زمان تامین می کند. اولین دلیل تحمل خطا است. اگر یک سرور یا حتی یک مرکز داده از کار بیفتد آنوقت نسخه های دیگر داده همچنان در دسترس خواهند بود و سیستم می تواند به کار خود ادامه دهد. دلیل دوم افزایش availability است یعنی کاربر حتی در صورت خرابی بخشی از سیستم هنوز بتواند داده را بخواند یا بنویسد. دلیل سوم بهبود کارایی و کاهش latency است. اگر نسخه ای از داده در نزدیکی کاربر قرار داشته باشد آنوقت عملیات read سریع تر انجام می شود. علاوه بر این replication می تواند بار ترافیک را میان چند سرور تقسیم کند و باعث شود سیستم تعداد درخواست های بیشتری را تحمل کند. در برخی سیستم ها replication همچنین برای مقیاس پذیری read-heavy workloads استفاده می شود، چون خواندن از چند replica باعث می شود گلوگاه از بین برود.

تفاوت replication برای افزایش کارایی و replication برای تحمل خطا چیست؟

Replication برای افزایش کارایی معمولا به این دلیل انجام می شود که درخواست های خواندن میان چند نسخه پخش شوند. در این حالت هدف اصلی این است که سرعت پاسخ گویی بهتر شود و throughput بالا برود. مثلا اگر داده ای روی چند replica باشد، کاربران می توانند از نزدیک ترین replica بخوانند و سیستم فشار را میان چند سرور تقسیم می کند. اما replication برای تحمل خطا هدف متفاوتی دارد. در اینجا اصل مهم این است که اگر یک replica خراب شد نسخه های دیگر همچنان داده را نگه دارند و سرویس قطع نشود. پس تمرکز اصلی روی reliability و fault tolerance است، نه فقط سرعت. البته در عمل این دو هدف اغلب هم زمان دنبال می شوند ولی طراحی سیستم برای آن ها ممکن است متفاوت باشد. مثلا سیستمی که فقط برای performance replicate شده است ممکن است از نسخه هایی با کمی inconsistency استفاده کند، اما سیستمی که برای fault tolerance ساخته شده معمولا به دنبال ضمانت های قوی تری درباره هماهنگی داده ها است.

مشکل اصلی replication از نظر consistency چیست؟

مشکل اصلی replication این است که وقتی داده در چند جا کپی می شود آنوقت این نسخه ها ممکن است هم زمان و کاملا یکسان نباشند. اگر یک client داده ای را در یک replica بنویسد، replica های دیگر ممکن است فورا از آن تغییر باخبر نشوند. در این فاصله یک client دیگر ممکن است از replica دیگری بخواند و مقدار قدیمی را ببیند. این وضعیت همان ناسازگاری موقت میان replica ها است. هر چه تعداد replica ها بیشتر باشد و فاصله ی جغرافیایی آن ها بیشتر شود آنوقت هماهنگ نگه داشتنشان سخت تر می شود. مخصوصا در حضور تاخیر شبکه، packet loss یا partition شدن شبکه ممکن است بعضی replica ها نتوانند به موقع با هم sync شوند. بنابراین replication همیشه یک trade-off بین سرعت، availability و consistency ایجاد می کند. مسئله اصلی این است که آیا سیستم باید همیشه آخرین مقدار را به همه نشان دهد، یا اجازه دهد برای مدتی نسخه های متفاوت وجود داشته باشند؟

تفاوت strong consistency و eventual consistency چیست؟

Strong consistency یعنی پس از انجام یک write موفق، همه ی readهای بعدی باید همان مقدار جدید را ببینند انگار که فقط یک نسخه از داده وجود دارد. در بسیاری از توضیحات strong consistency را معادل linearizability در نظر می گیرند یعنی عملیات ها طوری دیده می شوند که گویی در یک لحظه ی واقعی و در یک ترتیب جهانی انجام شده اند. این مدل برای برنامه نویس ساده تر است چون رفتار سیستم به یک shared memory واحد شبیه می شود. اما رسیدن به این مدل معمولا هزینه ی بیشتری دارد چون باید بین replica ها هماهنگی دقیق برقرار شود. در مقابل eventual consistency می گوید اگر write های جدید دیگری انجام نشوند، نسخه های مختلف داده در نهایت به یک مقدار مشترک می رسند. در این مدل سیستم اجازه می دهد برای مدتی replica ها مقدارهای متفاوتی داشته باشند. یعنی consistency فوری تضمین نمی شود اما فرض بر این است که با گذشت زمان و انتقال update ها، همه ی نسخه ها همگرا می شوند. این مدل معمولا availability و performance بالاتری می دهد ولی ممکن است کاربر در لحظه داده ی قدیمی بخواند. بنابراین eventual consistency برای کاربردهایی مناسب است که کمی تاخیر در انتشار تغییرات قابل قبول باشد.

مفهوم read-your-writes consistency چیست؟

Read-your-writes consistency یکی از مدل های consistency از دید client است. این مدل تضمین می کند که اگر یک client عملیاتی برای نوشتن انجام داده باشد، هر read بعدی توسط همان client باید حداقل همان write خودش را ببیند. به بیان ساده کاربر نباید بعد از ثبت تغییر خودش دوباره نسخه ی قدیمی را مشاهده کند. این ویژگی برای تجربه ی کاربری بسیار مهم است. مثلا اگر کاربر پروفایلش را تغییر دهد، انتظار دارد بلافاصله بعد از آن همان تغییر را در صفحه ی خود ببیند، حتی اگر سیستم در پشت صحنه هنوز همه replicaها را هماهنگ نکرده باشد. read-your-writes الزام نمی کند که همه ی کاربران فورا همان مقدار را ببینند بلکه فقط تضمین می کند که خود نویسنده تغییرات خودش را از دست ندهد.

مفهوم monotonic reads چیست؟

Monotonic reads هم یکی دیگر ازconsistency های client-centric است. این مدل تضمین می کند که اگر یک client یک بار مقدار جدیدی از داده را دیده باشد، درread های بعدی همان client نباید به مقدار قدیمی تر برگردد. یعنی مشاهده های client باید به صورت یک طرفه و غیرکاهشی پیش بروند. مثالا کاربر یک بار نسخه ی جدیدی از یک رکورد را در یک replica دیده است. در مدل monotonic reads، در درخواست بعدی نباید به replica دیگری هدایت شود که هنوز نسخه ی قدیمی را دارد و باعث شود کاربر یک وضعیت عقب رفته را ببیند. این ویژگی از نظر تجربه ی کاربری مهم است چون از دید کاربر منطقی نیست که سیستم به عقب برگردد. این مدل به خصوص در سیستم هایی که read ها ممکن است بین replica های مختلف جابه جا شوند اهمیت دارد.

چرا در سیستم های توزیع شده نمی توان همزمان بیشترین consistency و availability و partition toleranceرا به صورت کامل داشت؟

این سوال به قضیه CAP مربوط است. CAP می گوید در یک سیستم توزیع شده هنگام وقوع partition شبکه، نمی توان هم زمان هم Consistency کامل داشت، هم Availability کامل و هم Partition Tolerance کامل. چون partition در دنیای واقعی اجتناب ناپذیر است، سیستم باید در شرایط جدایی شبکه بین دو انتخاب

سخت یکی را ترجیح دهد. یا consistency را قربانی کند تا به درخواست ها جواب بدهد یا availability را قربانی کند تا داده ناسازگار نشود. مثلا شبکه بین دو replica قطع شده است. اگر هر دو طرف همچنان به درخواست ها پاسخ دهند آنوقت ممکن است داده ها ناسازگار شوند پس consistency کامل از بین می رود. اگر یکی از طرفین برای حفظ consistency از پاسخ دادن خودداری کند آنگاه سیستم دیگر fully available نیست. بنابراین در زمان partition نمی توان هم زمان هر دو ویژگی را به حداکثر رساند. نکته مهم این است که CAP نمی گوید همیشه فقط دو تا از این سه ویژگی را می توان داشت بلکه می گوید در زمان partition باید بین consistency و availability انتخاب کرد. به همین دلیل سیستم های واقعی معمولا بین این ویژگی ها trade-off برقرار می کنند. برخی سیستم ها consistency قوی تری می دهند و بعضی دیگر availability بالاتر و consistency ضعیف تر. انتخاب درست به نیاز برنامه بستگی دارد. مثلا در بانکداری consistency اهمیت بیشتری دارد اما در شبکه های اجتماعی یاcache ها eventual consistency اغلب قابل قبول است.

بخش دوم: پیاده سازی Store Key-Value Replicated

مقدمه

در این بخش، یک سیستم ساده Replicated Key-Value Store پیاده سازی شد. هدف از این بخش این بود که داده ها به جای ذخیره شدن روی یک سرور واحد، روی چند replica مستقل نگه داری شوند و رفتار سیستم در دو مدل consistency مختلف بررسی شود. سیستم پیاده سازی شده شامل سه replica مستقل است که هر کدام به صورت یک سرویس جداگانه اجرا می شوند. هر replica داده های مخصوص خود را به صورت محلی نگه داری می کند و با replica های دیگر از طریق شبکه ارتباط برقرار می کند.

در این پیاده سازی از زبان Go و پروتکل HTTP استفاده شده است. هر replica یک HTTP server مستقل است و روی یک پورت جداگانه اجرا می شود. سه replica سیستم به صورت زیر هستند:

Replica 1 روی پورت 8001

Replica 2 روی پورت 8002

Replica 3 روی پورت 8003

نکته مهم در معماری این سیستم این است که replica ها فقط object یا goroutine داخل یک برنامه واحد نیستند، بلکه هر کدام به صورت یک فرآیند مستقل اجرا می شوند. بنابراین برای اجرای کامل سیستم، باید سه terminal جدا باز شود و در هر terminal یکی از replica ها اجرا شود. این موضوع باعث می شود ارتباط بین replica ها واقعا از طریق شبکه و با درخواست های HTTP انجام شود.

معماری کلی سیستم

سیستم از دو بخش اصلی تشکیل شده است:

1. بخش replica

این بخش شامل کد اصلی سرورهای replica است. هر replica عملیات های GET و PUT را پشتیبانی می کند و همچنین می تواند پیام های replication را از replica های دیگر دریافت کند. هر replica یک فایل config جداگانه دارد که در آن شناسه replica ، شماره پورت، آدرس replica های دیگر، مقدار تاخیر مصنوعی replication و مسیر فایل ذخیره داده مشخص شده است.

2. بخش client

این بخش برای ارسال درخواست به replica ها استفاده می شود. کلاینت می تواند به هر replica دلخواه درخواست PUT یا GET بفرستد. همچنین در زمان ارسال درخواست، می توان مشخص کرد که عملیات در حالت eventual consistency انجام شود یا در حالت strong consistency .

ساختار کلی پروژه به صورت زیر است:

```
HW3/
replica/
main.go
client/
main.go
configs/
replica1.json
replica2.json
```

replica3.json
results/

در پوشه replica ، کد مربوط به اجرای replica ها قرار دارد. در پوشه client ، کد مربوط به کلاینت قرار دارد. در پوشه configs ، فایل های تنظیمات مربوط به هر replica قرار داده شده است. در پوشه results نیز فایل های داده محلی هر replica ذخیره می شوند.

نحوه اجرای replica ها

برای اجرای سیستم، ابتدا باید وارد پوشه اصلی پروژه شویم. سپس باید سه terminal جدا باز کنیم. در terminal اول، Replica 1 را اجرا می کنیم:

```
go run replica/main.go -config configs/replica1.json
```

در terminal دوم، Replica 2 را اجرا می کنیم:

```
go run replica/main.go -config configs/replica2.json
```

در terminal سوم، Replica 3 را اجرا می کنیم:

```
go run replica/main.go -config configs/replica3.json
```

یک نمونه از اجرای اینها به صورت زیر است:

```
PS D:\Dist\Projects\HW3> go run replica/main.go -config configs/replica1.json
2026/06/15 10:40:28 replica1 is running on :8001
2026/06/15 10:40:28 peers: [http://localhost:8002 http://localhost:8003]
2026/06/15 10:40:28 replication delay: 500 ms
```

شکل 1. نمونه ای از اجرای replica 1

بعد از اجرای این سه دستور، هر سه replica فعال هستند و می توانند درخواست های کلاینت و همچنین درخواست های replication از replica های دیگر را دریافت کنند. برای مثال، Replica 1 روی آدرس زیر در دسترس است:

http://localhost:8001

Replica 2 روی آدرس زیر اجرا می شود:

http://localhost:8002

و Replica 3 روی آدرس زیر اجرا می شود:

http://localhost:8003

در این سیستم برای آدرس اصلی / صفحه وب طراحی نشده است، چون replica ها نقش API server دارند نه وب سایت. بنابراین اگر آدرس اصلی مثل http://localhost:8001 در مرورگر باز شود، ممکن است خطای 404 نمایش داده شود. این خطا به معنای اشتباه بودن اجرا نیست بلکه فقط یعنی برای مسیر / صفحه ای تعریف نشده است. برای مشاهده داده های هر replica باید از مسیر dump/ استفاده شود، مثلا:

http://localhost:8001/dump

http://localhost:8002/dump

http://localhost:8003/dump

نحوه اجرای client

برای ارسال درخواست به replica ها از برنامه client استفاده می شود. کلاینت می تواند عملیات PUT ، GET و dump را انجام دهد.

برای نوشتن یک مقدار در حالتeventual consistency ، دستور زیر استفاده می شود:

```
go run client/main.go -op put -replica http://localhost:8001 -key x -value 10 -mode eventual
```

این دستور مقدار x=10 را به Replica 1 ارسال می کند. چون mode برابر eventual است، Replica 1 ابتدا مقدار را در داده محلی خودش ذخیره می کند و سپس به صورت غیرهمزمان آن را به replica های دیگر ارسال می کند.

برای خواندن مقدار x از Replica 2 در حالت eventual consistency ، از دستور زیر استفاده می شود:

```
go run client/main.go -op get -replica http://localhost:8002 -key x -mode eventual
```

اگر این دستور بلافاصله بعد از PUT اجرا شود، ممکن است Replica 2 هنوز مقدار جدید را دریافت نکرده باشد. در این حالت ممکن است مقدار قدیمی یا پیام پیدا نشدن key برگردد (که سناریو های مختلف را با تصویر در

بخش چهارم بررسی می کنیم). اما پس از گذشت چند لحظه و تکمیل شدن replication ، اگر دوباره همان دستور اجرا شود، مقدار جدید x=10 از Replica 2 نیز خوانده می شود.

برای نوشتن مقدار در حالت strong consistency ، از دستور زیر استفاده می شود:

```
go run client/main.go -op put -replica http://localhost:8001 -key y -value 20 -mode strong
```

در این حالت، عملیات PUT فقط زمانی موفق محسوب می شود که حداقل اکثریت replica ها مقدار جدید را تایید کنند. چون در این پروژه سه replica وجود دارد، اکثریت برابر با دو replica است. بنابراین اگر Replica 1 مقدار را روی خودش ذخیره کند و حداقل یکی از دو replica دیگر نیز مقدار را تایید کند، عملیات موفق خواهد بود.

برای خواندن مقدار y در حالت strong consistency می توان از دستور زیر استفاده کرد:

```
go run client/main.go -op get -replica http://localhost:8002 -key y -mode strong
```

در حالت strong ، خواندن فقط به مقدار محلی همان replica محدود نمی شود. replica دریافت کننده درخواست، مقدار key را از replica های دیگر نیز بررسی می کند و جدیدترین مقدار را برمی گرداند. به این ترتیب احتمال برگشت دادن مقدار قدیمی کاهش پیدا می کند.

برای مشاهده تمام داده های ذخیره شده در یک replica نیز از دستور dump استفاده می شود:

```
go run client/main.go -op dump -replica http://localhost:8001
```

یا از طریق مرورگر:

http://localhost:8001/dump

توضیح عملیات PUT

عملیات PUT برای نوشتن یا به روز رسانی مقدار یک key استفاده می شود. وقتی کلاینت یک درخواست PUT به یکی از replica ها می فرستد، آن replica ابتدا درخواست را دریافت می کند و سپس بسته به مدل consistency انتخاب شده، رفتار متفاوتی دارد.

اگر مدل انتخاب شده eventual باشد، replica دریافت کننده مقدار جدید را فورا در داده محلی خود ذخیره می کند و سریع به کلاینت پاسخ موفق می دهد. سپس عملیات ارسال مقدار جدید به replica های دیگر در پس زمینه انجام می شود. در نتیجه پاسخ PUT سریع تر است اما ممکن است برای مدت کوتاهی replica های دیگر مقدار جدید را نداشته باشند.

اگر مدل انتخاب شده strong باشد، replica دریافت کننده فقط زمانی عملیات PUT را موفق اعلام می کند که اکثریت replica ها مقدار جدید را تایید کرده باشند. در این حالت، replica دریافت کننده مقدار را به replica های دیگر می فرستد و منتظر پاسخ آن ها می ماند. اگر حداقل دو replica از سه replica مقدار را داشته باشند، عملیات موفق می شود. اگر اکثریت در دسترس نباشد، عملیات شکست می خورد.

توضیح عملیات GET

عملیات GET برای خواندن مقدار یک key استفاده می شود. در حالت eventual consistency ، replica فقط مقدار محلی خودش را بررسی می کند. بنابراین اگر replica هنوز پیام replication مربوط به یک PUT جدید را دریافت نکرده باشد، ممکن است مقدار قدیمی را برگرداند. این رفتار نشان دهنده stale read است و در مدل eventual consistency قابل انتظار است.

در حالت strong consistency ، خواندن به شکل هماهنگ شده تری انجام می شود .دریافت کننده درخواست تلاش می کند مقدار key را از replica های دیگر نیز بگیرد و جدیدترین مقدار را انتخاب کند. بنابراین در این حالت احتمال اینکه کلاینت مقدار قدیمی دریافت کند کمتر است. این روش باعث افزایش consistency می شود، اما ممکن است زمان پاسخ GET را بیشتر کند، چون replica باید با replica های دیگر نیز ارتباط برقرار کند.

توضیح مدل Eventual Consistency

در مدل eventual consistency ، تمرکز اصلی روی دسترس پذیری و سرعت پاسخ است. وقتی یک PUT انجام می شود، replica دریافت کننده مقدار را فورا ذخیره می کند و بدون منتظر ماندن برای replica های دیگر به کلاینت پاسخ می دهد. سپس مقدار جدید به صورت غیرهمزمان برای replica های دیگر ارسال می شود.

مزیت این مدل این است که عملیات نوشتن سریع انجام می شود و حتی اگر بعضی replica ها کند یا موقتا در دسترس نباشند، replica دریافت کننده می تواند به درخواست پاسخ دهد. اما مشکل این مدل این است که برای مدت کوتاهی replica ها ممکن است مقدارهای متفاوتی داشته باشند. مثلا اگر مقدار x=10 روی Replica 1 نوشته شود و بلافاصله از Replica 2 خوانده شود، ممکن است Replica 2 هنوز مقدار x=10 را نداشته باشد. بعد از رسیدن پیام replication ، Replica 2 نیز به روز می شود و در نهایت همه replica ها به مقدار یکسان می رسند.

بنابراین در این مدل، consistency بلافاصله تضمین نمی شود اما سیستم در نهایت همگرا می شود. این رفتار برای مشاهده مفهوم ناسازگاری موقت بین replica ها بسیار مناسب است.

توضیح مدل Strong Consistency

در مدل strong consistency ، عملیات PUT فقط زمانی موفق است که اکثریت replica ها مقدار جدید را تایید کنند. در این پروژه چون تعداد replica ها سه عدد است، اکثریت یعنی حداقل دوreplica . بنابراین اگر کلاینت به Replica 1 درخواست PUT بدهد، Replica 1 تلاش می کند مقدار جدید را به Replica 2 و Replica 3 نیز ارسال کند. اگر حداقل یکی از آن ها پاسخ موفق بدهد، همراه با خود Replica 1 اکثریت تشکیل می شود و عملیات موفق خواهد بود.

مزیت این مدل این است که احتمال ناسازگاری بین replica ها کمتر می شود و پس از موفق شدنPUT ، مقدار جدید حداقل در اکثریت سیستم وجود دارد. اما هزینه این مدل افزایش زمان پاسخ و کاهش availability در زمان خرابی است. چون replica دریافت کننده باید منتظر تایید replica های دیگر بماند، عملیات PUT نسبت به eventual consistency کندتر می شود. همچنین اگر اکثریت replica ها در دسترس نباشند، عملیات PUT شکست می خورد.

ارتباط شبکه ای بین replica ها

یکی از نیازمندی های مهم پروژه این بود که ارتباط بین replica ها واقعا از طریق شبکه انجام شود و همه عملیات ها فقط روی یک متغیر مشترک داخل یک برنامه انجام نشوند. در این پیاده سازی، هر replica یک HTTP server مستقل است و برای ارسال داده به replica های دیگر از درخواست HTTP استفاده می کند.

برای مثال، وقتی Replica 1 می خواهد مقدار جدید را به Replica 2 ارسال کند، یک درخواست HTTP به مسیر داخلی Replica 2 می فرستد:

http://localhost:8002/internal/replicate

این مسیر مخصوص ارتباط داخلی بین replica هاست. کلاینت عادی از این مسیر استفاده نمی کند. Replica دریافت کننده با گرفتن این پیام، مقدار جدید را بررسی می کند و در صورت معتبر بودن، آن را در داده محلی خود ذخیره می کند.

نگه داری داده به صورت محلی

هر replica داده های خودش را به صورت محلی ذخیره می کند. داده ها در حافظه برنامه نگه داری می شوند و همچنین در فایل JSON مربوط به همان replica ذخیره می شوند. برای مثال، داده های Replica 1 در فایلی مانند مسیر زیر ذخیره می شود:

results/replica1_data.json

داده های Replica 2 و Replica 3 نیز در فایل های جداگانه ذخیره می شوند. این طراحی باعث می شود هر replica واقعا state مستقل خودش را داشته باشد. بنابراین اگر یک replica مقدار جدید را هنوز دریافت نکرده باشد، داده محلی آن با replica های دیگر متفاوت خواهد بود و این موضوع امکان مشاهده ناسازگاری موقت را فراهم می کند.

تاخیر مصنوعی در replication

برای شبیه سازی تاخیر شبکه، در فایل config هر replica یک مقدار به نام replication_delay_ms قرار داده شده است. این مقدار مشخص می کند replica قبل از ارسال پیام replication به replica های دیگر چند میلی ثانیه صبر کند.

برای مثال، اگر مقدار آن برابر 500 باشد، پیام replication با تاخیر 500 میلی ثانیه ارسال می شود. این قابلیت برای بررسی تاثیر تاخیر شبکه روی consistency مفید است. اگر این مقدار زیاد شود، بازه زمانی که در آن

replicaها مقدارهای متفاوتی دارند بیشتر می شود و احتمال stale read در مدل eventual consistency افزایش پیدا می کند.

توضیح کلی کد پیاده سازی

کد پروژه به دو قسمت اصلی تقسیم شده است: کد replica و کدclient . در قسمت replica ، منطق اصلی سیستم پیاده سازی شده است. هر replica هنگام اجرا، فایل تنظیمات مخصوص خود را می خواند و بر اساس آن شناسه خود، شماره پورت، آدرس replica های دیگر، مسیر فایل ذخیره داده و مقدار تاخیر replication را مشخص می کند. سپس به صورت یک HTTP server مستقل اجرا می شود و آماده دریافت درخواست از client یا replica های دیگر می شود.

هر replica یک ساختار داده محلی برای نگه داری key و value ها دارد. علاوه بر مقدار اصلی، برای هر داده اطلاعاتی مثل نسخه، شناسه replica به روز کننده و زمان به روز رسانی نیز ذخیره می شود. این اطلاعات باعث می شود replica بتواند تشخیص دهد که یک مقدار دریافتی جدیدتر است یا قدیمی تر. همچنین اگر دو مقدار متفاوت برای یک key به وجود بیاید، سیستم بتواند بر اساس سیاست مشخص شده یکی از آن ها را انتخاب کند. در کد replica چند مسیر اصلی تعریف شده است. مسیر PUT برای نوشتن یا به روز رسانی مقدار یک key استفاده می شود. مسیر GET برای خواندن مقدار یک key است. مسیر dump برای مشاهده کل داده های موجود در یک replica استفاده می شود. همچنین چند مسیر داخلی نیز وجود دارد که مخصوص ارتباط بین replica ها هستند. مثلا وقتی یک replica می خواهد مقدار جدید را به replica های دیگر ارسال کند، از مسیر داخلی replication استفاده می کند.

در عملیات PUT ، ابتدا replica دریافت کننده درخواست، مقدار جدید را می سازد و نسخه آن را افزایش می دهد. سپس بسته به مدل consistency انتخاب شده، رفتار متفاوتی انجام می شود. در حالت eventual consistency، مقدار ابتدا روی همان replica ذخیره می شود و پاسخ موفق سریع به client برگردانده می شود. سپس ارسال مقدار به replica های دیگر در پس زمینه انجام می شود. به همین دلیل ممکن است برای مدت کوتاهی بعضی replica ها مقدار جدید را نداشته باشند.

در حالت strong consistency ، replica دریافت کننده فقط زمانی عملیات را موفق اعلام می کند که حداقل اکثریت replica ها مقدار جدید را تایید کرده باشند. چون در این پروژه سه replica وجود دارد، اکثریت

برابر با دو replica است. بنابراین اگر مقدار روی replica دریافت کننده و حداقل یک replica دیگر ثبت شود، عملیات موفق محسوب می شود. در غیر این صورت، عملیات PUT شکست می خورد.

در عملیات GET نیز دو رفتار وجود دارد. در حالت eventual consistency ، replica فقط داده محلی خودش را بررسی می کند و همان را برمی گرداند. بنابراین اگر هنوز replication کامل نشده باشد، ممکن است مقدار قدیمی دیده شود. اما در حالت strong consistency ، replica تلاش می کند مقدار key را از replica های دیگر نیز بررسی کند و جدیدترین نسخه را برگرداند. این کار باعث می شود احتمال خواندن مقدار قدیمی کمتر شود.

در قسمت client ، یک برنامه ساده command line نوشته شده است که به کمک آن می توان عملیات PUT، GET و dump را روی هر replica دلخواه اجرا کرد. کاربر هنگام اجرای client مشخص می کند که عملیات روی کدام replica انجام شود، key و value چه باشند و مدل consistency برابر eventual یا strong باشد. این client باعث می شود آزمایش کردن رفتار سیستم در سناریوهای مختلف ساده تر شود. به طور کلی، کد به گونه ای نوشته شده است که هر replica مستقل باشد، داده خود را جداگانه نگه دارد و ارتباط بین replica ها از طریق HTTP انجام شود. بنابراین سیستم فقط یک شبیه سازی ساده درون یک برنامه واحد نیست، بلکه یک پیاده سازی شبکه ای ساده از Replicated Key-Value Store است.

جمع بندی بخش پیاده سازی

در این بخش یک Replicated Key-Value Store ساده با سه replica مستقل پیاده سازی شد. هر replica به صورت یک سرویس جداگانه اجرا می شود، داده های خود را محلی نگه داری می کند و از طریق HTTP با replica های دیگر ارتباط دارد. کلاینت می تواند به هر replica درخواست GET و PUT بفرستد و نوع consistency را مشخص کند.

در مدل eventual consistency ، نوشتن ابتدا روی replica دریافت کننده انجام می شود و سپس به صورت غیرهمزمان به replica های دیگر منتقل می شود. این مدل سریع تر است و availability بیشتری دارد اما ممکن است برای مدت کوتاهی stale read رخ دهد.

در مدل strong consistency ساده شده، عملیات PUT فقط در صورتی موفق است که حداقل اکثریت replicaها مقدار جدید را تایید کنند. این مدل consistency بهتری فراهم می کند، اما زمان پاسخ آن بیشتر است و در صورت در دسترس نبودن اکثریت replica ها، عملیات نوشتن شکست می خورد.

بنابراین این پیاده سازی نشان می دهد که replication علاوه بر افزایش دسترس پذیری و تحمل خطا، چالش consistency را نیز ایجاد می کند. همچنین با مقایسه دو مدل eventual و strong می توان مشاهده کرد که در سیستم های توزیع شده معمولا بین سرعت، availability و consistency یک trade-off وجود دارد.

بخش سوم: نسخه گذاری و حل تعارض

مقدمه

در این بخش، برای سیستم Replicated Key-Value Store یک مکانیزم نسخه گذاری و حل تعارض پیاده سازی شد. دلیل نیاز به نسخه گذاری این است که در یک سیستم replicated ، هر داده روی چند replica مختلف ذخیره می شود و ممکن است به دلیل تاخیر شبکه، خرابی یک replica یا انجام همزمان عملیات ها، replica های مختلف برای یک کلید مقدارهای متفاوتی داشته باشند. اگر فقط مقدار key و value ذخیره شود، replica ها نمی توانند تشخیص دهند کدام مقدار جدیدتر است و ممکن است یک مقدار قدیمی به اشتباه جایگزین مقدار جدیدتر شود. برای جلوگیری از این مشکل، برای هر مقدار یک شماره نسخه در نظر گرفته شد.

جزئیات پیاده سازی

در این پیاده سازی، هر رکورد ذخیره شده فقط شامل key و value نیست بلکه اطلاعات بیشتری نیز همراه آن نگه داری می شود. ساختار کلی هر داده شامل key ، value، version، updated_by و updated_at است. فیلد key نام کلید را مشخص می کند. فیلد value مقدار ذخیره شده برای آن کلید است. فیلد version شماره نسخه مقدار را نشان می دهد. فیلد updated_by مشخص می کند آخرین بار کدام replica این مقدار را نوشته یا به روز رسانی کرده است. فیلد updated_at نیز زمان به روز رسانی را نگه داری می کند.

برای مثال، اگر مقدار x=10 توسط Replica 1 نوشته شود، رکورد آن به شکل زیر خواهد بود:

```
{
"key": "x",
"value": "10",
```

```
"version": 1,
"updated_by": "replica1",
"updated_at": "زمان به روز رسانی"
}
```

در این ساختار، مهم ترین قسمت برای تشخیص جدیدتر بودن داده، فیلد version است. هر بار که مقدار یک key با عملیات PUT تغییر می کند، نسخه آن یک واحد افزایش می یابد. اگر کلید برای اولین بار نوشته شود، نسخه آن برابر 1 قرار داده می شود. اگر همان کلید قبلا وجود داشته باشد، نسخه جدید برابر نسخه قبلی به علاوه یک خواهد بود. برای مثال، اگر مقدار x ابتدا برابر 10 و دارای version برابر 1 باشد، با انجام PUT جدید و تغییر مقدار x به 20، نسخه آن به 2 افزایش پیدا می کند.

این روش باعث می شود replica ها هنگام دریافت پیام replication بتوانند تشخیص دهند که مقدار دریافتی جدیدتر است یا قدیمی تر. اگر یک replica رکوردی دریافت کند که version آن از version محلی بیشتر باشد، آن رکورد جدیدتر محسوب می شود و مقدار محلی با مقدار جدید جایگزین می گردد. اما اگر رکورد دریافتی version کمتری نسبت به مقدار محلی داشته باشد، یعنی پیام دریافتی قدیمی است و نباید اعمال شود. در این حالت replica مقدار جدیدتر خودش را حفظ می کند و پیام قدیمی را نادیده می گیرد.

این موضوع برای جلوگیری از overwrite شدن مقدار جدید توسط مقدار قدیمی بسیار مهم است. فرض کنید Replica 1 مقدار زیر را برای کلید x داشته باشد:

x = 20, version = 2

حال اگر به دلیل تاخیر شبکه، یک پیام قدیمی تر به Replica 1 برسد که در آن مقدار x برابر 10 و version برابر 1 است، replica نباید مقدار x=20 را با x=10 جایگزین کند. چون version مقدار دریافتی کمتر است، replica تشخیص می دهد که این پیام قدیمی است و آن را اعمال نمی کند. به این ترتیب، versioning مانع خراب شدن داده های جدیدتر توسط پیام های قدیمی می شود.

علاوه بر نسخه گذاری، در این بخش باید حالتی نیز مدیریت شود که دو مقدار متفاوت برای یک key با version یکسان ایجاد شوند. این حالت معمولا زمانی رخ می دهد که دو replica مختلف تقریبا به صورت همزمان برای یک کلید عملیات PUT انجام دهند. برای مثال ممکن است در یک بازه زمانی کوتاه، Replica 1 مقدار x=10 را بنویسد و Replica 2 نیز تقریبا همزمان مقدار x=20 را بنویسد. اگر هر دو replica قبل از دریافت پیام

replication از یکدیگر این عملیات را انجام دهند، هر دو مقدار ممکن است version برابر 1 داشته باشند اما value آن ها متفاوت باشد.

در این وضعیت، فقط با مقایسه version نمی توان تصمیم گرفت کدام مقدار باید نهایی شود زیرا version هر دو مقدار برابر است. بنابراین باید یک سیاست مشخص برای حل تعارض یا conflict resolution وجود داشته باشد. در این پیاده سازی، سیاست انتخاب شده به این صورت است که اگر دو رکورد version یکسان ولی value متفاوت داشته باشند، رکوردی انتخاب می شود که شناسه replica به روز کننده آن بزرگ تر باشد. یعنی اگر بین مقدارهای نوشته شده توسط replica های مختلف تعارض ایجاد شود، ترتیب اولویت به شکل زیر است:

replica3 > replica2 > replica1

بنابراین اگر برای یک کلید، Replica 1 مقدار 10 و Replica 2 مقدار 20 را با version یکسان تولید کرده باشند، مقدار مربوط به Replica 2 به عنوان مقدار نهایی انتخاب می شود. اگر تعارض بین Replica 2 و Replica 3 باشد، مقدار مربوط به Replica 3 برنده خواهد شد. این سیاست ساده است اما یک ویژگی مهم دارد: قطعی و قابل تکرار است. یعنی همه replica ها با دریافت همان رکوردها به یک نتیجه یکسان می رسند و در نهایت سیستم همگرا می شود.

در این پیاده سازی، ترتیب تصمیم گیری برای انتخاب مقدار بهتر به این صورت است: ابتدا version مقایسه می شود. اگر version یک رکورد بیشتر باشد، همان رکورد جدیدتر است و انتخاب می شود. اگر version ها برابر باشند، شناسه replica به روز کننده مقایسه می شود و رکورد مربوط به replica با شناسه بزرگ تر انتخاب می گردد. اگر حتی شناسه replica نیز برابر باشد، زمان به روز رسانی جدیدتر به عنوان معیار نهایی استفاده می شود.

این سیاست باعث می شود سه حالت اصلی به درستی مدیریت شوند. حالت اول زمانی است که یک مقدار کاملا جدید برای یک key دریافت می شود. در این حالت، چون replica قبلا آن key را نداشته است، مقدار دریافتی ذخیره می شود. حالت دوم زمانی است که مقدار دریافتی version بالاتری نسبت به مقدار محلی دارد. در این حالت مقدار دریافتی جدیدتر است و جایگزین مقدار قبلی می شود. حالت سوم زمانی است که مقدار دریافتی versionپایین تری دارد. در این حالت مقدار دریافتی قدیمی است و نادیده گرفته می شود. اگر version برابر ولی value متفاوت باشد، conflict رخ داده و سیاست حل تعارض اجرا می شود.

استفاده از versioning و conflict resolution در این پروژه باعث شد سیستم بتواند با پیام های دیر رسیده، داده های قدیمی و نوشتن های همزمان برخورد درستی داشته باشد. بدون این مکانیزم ها، ممکن بود مقدار جدیدتر به دلیل دریافت یک پیام قدیمی از بین برود یا replica های مختلف بعد از یک تعارض همزمان برای همیشه مقدارهای متفاوتی داشته باشند. اما با این طراحی، هر replica می تواند درباره رکوردهای دریافتی تصمیم بگیرد و در نهایت همه replica ها به یک مقدار مشترک برسند.

در مدل eventual consistency ، اهمیت versioning بیشتر دیده می شود، زیرا replication به صورت غیرهمزمان انجام می شود و ممکن است پیام ها با تاخیر به replica های دیگر برسند. در چنین شرایطی، ممکن است یک replica ابتدا مقدار جدیدتر را دریافت کند و بعدا یک پیام قدیمی تر به آن برسد. با استفاده از version، replica می تواند پیام قدیمی تر را تشخیص دهد و آن را نادیده بگیرد. همچنین در صورت ایجاد تعارض همزمان، سیاست conflict resolution باعث می شود replica ها پس از تبادل پیام های replication به یک مقدار نهایی یکسان برسند.

در مدل strong consistency نیز versioning مفید است زیرا هنگام خواندن یا نوشتن هماهنگ شده، replica ها می توانند نسخه های مختلف یک key را مقایسه کنند و جدیدترین نسخه را تشخیص دهند. به این ترتیب، حتی اگر یک replica از بقیه عقب مانده باشد، سیستم می تواند مقدار جدیدتر را پیدا کند و در صورت نیاز replica عقب مانده را اصلاح کند.

جمع بندی

به طور کلی، در این بخش یک روش ساده اما کاربردی برای مدیریت نسخه ها و تعارض ها پیاده سازی شد. هر بار که مقدار یک کلید تغییر می کند، version آن افزایش می یابد. هر replica هنگام دریافت یک رکورد از replica دیگر، version آن را با مقدار محلی مقایسه می کند. مقدار جدیدتر جایگزین می شود، مقدار قدیمی تر نادیده گرفته می شود و در صورت برابر بودن version و متفاوت بودنvalue ، سیاست حل تعارض بر اساس شناسه replica اجرا می شود. این طراحی باعث می شود داده های جدیدتر حفظ شوند، مشکل تعارض های همزمان حل شود و replica ها در نهایت به وضعیت یکسان برسند.

بخش چهارم: آزمایش و تحلیل

سناریو 1: مشاهده ناسازگاری موقت

هدف این سناریو بررسی رفتار سیستم در مدل eventual consistency است. در این مدل، وقتی یک عملیات PUT روی یک replica انجام می شود، مقدار جدید ابتدا فقط روی همان replica اعمال می شود و سپس به صورت غیرهمزمان برای replica های دیگر ارسال می گردد. بنابراین انتظار داریم که اگر بلافاصله پس از PUT از یک replica دیگر عملیات GET انجام شود، ممکن است مقدار جدید هنوز در آن replica وجود نداشته باشد. اما پس از گذشت چند ثانیه و تکمیل شدن replication ، همه replica ها باید به مقدار یکسان برسند.

برای اجرای این سناریو، ابتدا هر سه replica به صورت مستقل اجرا شدند. سپس مقدار x=10 روی Replica 1 و در حالت eventual consistency نوشته شد (همچنین مقدار تاخیر ار هم 4000 میلی ثانیه گذاشتم که نتایج قابل مشاهده باشند):

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key x -value 10 -mode eventual
HTTP status: 200 OK
Client measured latency: 28.106 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "x",
    "value": "10",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781946204655547000
  },
  "updated_replicas": 1,
  "latency_ms": 18.749
}
```

شکل 2. اجرا در حالت eventual consistency روی replica1

همان طور که در شکل 2 مشاهده می شود، عملیات PUT با موفقیت انجام شد. مقدار key برابر x و value برابر 10 است. همچنین version مقدار برابر 1 و updated_by برابر replica1 ثبت شده است. در خروجی، مقدار updated_replicas برابر 1 است. این موضوع نشان می دهد که در لحظه پاسخ دادن به client ، مقدار فقط روی replica دریافت کننده یعنی Replica 1 اعمال شده است و replication به replica های دیگر هنوز به صورت کامل انجام نشده است. زمان پاسخ client برای این عملیات حدود 28.106 میلی ثانیه ثبت شد.

در مرحله بعد، بلافاصله پس از انجامPUT ، مقدار x از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key x -mode eventual
HTTP status: 404 Not Found
Client measured latency: 7.877 ms
{
  "ok": false,
  "message": "key not found",
  "record": {
    "key": "",
    "value": "",
    "version": 0,
    "updated_by": "",
    "updated_at": 0
  },
  "checked_replicas": 0,
  "latency_ms": 0
}
```

شکل 3. مقدار آپدیت نشده replica2 قبل از 4 ثانیه

همان طور که در شکل 3 دیده می شود، پاسخ Replica 2 برابر key not found بود و HTTP status برابر Not Found 404 برگشت. این نتیجه نشان می دهد که Replica 2 در آن لحظه هنوز پیام replication مربوط به مقدار جدید x=10 را دریافت نکرده بود. بنابراین Replica 2 هنوز مقدار جدید را نداشت. این دقیقا همان ناسازگاری موقت یا stale read در مدل eventual consistency است. زمان پاسخ client برای این GET حدود 7.877 میلی ثانیه بود.

سپس پس از گذشت چند ثانیه، دوباره مقدار x از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key x -mode eventual
HTTP status: 200 OK
Client measured latency: 8.370 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "x",
    "value": "10",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781946204655547000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 4. مقدار آپدیت شده replica2 قبل از 4 ثانیه

در شکل 4 مشاهده می شود که این بار Replica 2 مقدار x=10 را با موفقیت برگردانده است. مقدار version برابر 1 و updated_by برابر replica1 است. این یعنی مقدار ابتدا روی Replica 1 نوشته شده و سپس از طریق replication به Replica 2 رسیده است. زمان پاسخ client در این مرحله حدود 8.370 میلی ثانیه بود.

برای اطمینان بیشتر از همگرا شدن سیستم، مقدار x از Replica 3 نیز خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key x -mode eventual
HTTP status: 200 OK
Client measured latency: 9.116 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "x",
    "value": "10",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781946204655547000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 5. مقدار درست روی replica3 بعد از 4 ثانیه

همان طور که در شکل 5 دیده می شود، Replica 3 نیز مقدار x=10 را برگردانده است. بنابراین پس از گذشت چند ثانیه، مقدار جدید به replica های دیگر نیز رسیده و سیستم به وضعیت همگرا رسیده است.

تحلیل سناریو

نتیجه این سناریو نشان می دهد که در مدل eventual consistency ، پس از انجام PUT ، همه replica ها بلافاصله مقدار جدید را مشاهده نمی کنند. در این آزمایش، مقدار x=10 ابتدا فقط روی Replica 1 ذخیره شد و client پاسخ موفق دریافت کرد. اما بلافاصله پس از آن، خواندن از Replica 2 باعث برگشت key not found شد، زیرا Replica 2 هنوز پیام replication را دریافت نکرده بود. پس از گذشت چند ثانیه، Replica 2 و Replica 3 نیز مقدار جدید را دریافت کردند و مقدار x=10 را برگرداندند. بنابراین در این سناریو، ناسازگاری موقت بین replica ها مشاهده شد. با این حال، پس از تکمیل شدن replication ، همه replica ها به مقدار یکسان رسیدند. این رفتار نشان دهنده ویژگی اصلی eventual consistency است: سیستم ممکن است برای مدت کوتاهی ناسازگار باشد، اما اگر عملیات جدیدی انجام نشود و پیام های replication منتقل شوند، replica ها در نهایت همگرا می شوند.

متریک های ثبت شده در این سناریو

PUT latency: حدود 28.106 میلی ثانیه از دید client

GET latency فوری از Replica 2 : حدود 7.877 میلی ثانیه

GET latency پس از همگرایی از Replica 2 : حدود 8.370 میلی ثانیه

GET latency پس از همگرایی از Replica 3 : حدود 9.116 میلی ثانیه

تعداد replica های به روز شده در پاسخ اولیه :PUT برابر 1

تعداد stale read مشاهده شده: 1

زمان همگرایی مشاهده شده: پس از گذشت 4 ثانیه، مقدار جدید روی Replica 2 و Replica 3 نیز دیده شد

سناریو 2: خرابی یک replica

هدف این سناریو بررسی رفتار سیستم در زمان خرابی یا توقف یکی از replica ها است. در این آزمایش، ابتدا هر سه replica اجرا شدند و سپس Replica 3 متوقف شد تا خرابی یک replica شبیه سازی شود. پس از آن، رفتار سیستم در دو مدل eventual consistency و strong consistency بررسی شد. همچنین برای کامل تر شدن آزمایش، حالتی نیز بررسی شد که دو replica خاموش باشند و اکثریت در مدل strong consistency از بین برود.

در ابتدای آزمایش، Replica 3 که روی پورت 8003 اجرا شده بود، با استفاده از Ctrl + C متوقف شد. به این ترتیب، Replica 3 از دسترس خارج شد و فقط Replica 1 و Replica 2 فعال باقی ماندند.

```
PS D:\Dist\Projects\HW3> go run replica/main.go -config configs/replica3.json
2026/06/20 13:10:20 starting with empty data store: open results/replica3_data.json: The system cannot find the file spe
cified.
2026/06/20 13:10:20 replica3 is running on :8003
2026/06/20 13:10:20 peers: [http://localhost:8001 http://localhost:8002]
2026/06/20 13:10:20 replication delay: 4000 ms
exit status 0xc000013a
PS D:\Dist\Projects\HW3>
```

شکل 6. متوقف کردن اجرای replica3 (خراب شدن)

پس از خاموش شدن Replica 3 ، ابتدا مدل eventual consistency بررسی شد. در این حالت، یک عملیات PUT روی Replica 1 انجام شد و مقدار s2_eventual=100 نوشته شد.

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s2_eventual -value 100 -mode
eventual
HTTP status: 200 OK
Client measured latency: 23.080 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s2_eventual",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781948581499142300
  },
  "updated_replicas": 1,
  "latency_ms": 13.735
}
```

شکل 7. نوشتن رو replica1 به صورت eventual

همان طور که در شکل 7 مشاهده می شود، عملیات PUT با موفقیت انجام شد و وضعیت پاسخ HTTP برابر 200 OK بود. در خروجی، مقدار ok برابر true و پیام پاسخ برابر PUT applied locally; replication is running asynchronouslyاست. این نشان می دهد که در مدلeventual consistency ، مقدار ابتدا روی Replica 1 اعمال شده و سپس replication به صورت غیرهمزمان در پس زمینه انجام شده است. مقدار updated_replicas برابر 1 است، زیرا در لحظه پاسخ دادن بهclient ، مقدار فقط روی Replica 1 ذخیره شده بود. زمان پاسخ client در این مرحله حدود 23.080 میلی ثانیه ثبت شد.

سپس برای بررسی ذخیره شدن مقدار روی1 Replica ، عملیات GET از همان replica انجام شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8001 -key s2_eventual -mode eventual
HTTP status: 200 OK
Client measured latency: 7.858 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s2_eventual",
    "value": "100",
    "version": 2,
    "updated_by": "replica1",
    "updated_at": 1781948701426241100
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 8. مقدار ذخیره شده روی Replica 1 در حالت eventual

در شکل 8 مشاهده می شود که Replica 1 مقدار s2_eventual=100 را با موفقیت برگردانده است. همچنین versionمقدار برابر 2 و updated_by برابر replica1 است. این خروجی نشان می دهد که مقدار روی Replica 1ذخیره شده و خواندن محلی در حالت eventual consistency به درستی انجام شده است. زمان پاسخ client برای این خواندن حدود 7.858 میلی ثانیه بود.

در مرحله بعد، پس از گذشت چند ثانیه، مقدار s2_eventual از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s2_eventual -mode eventual
HTTP status: 200 OK
Client measured latency: 7.642 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s2_eventual",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781949017106378800
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 9. آپدیت شدن مقدار replica2 پس از 4 ثانیه

همان طور که در شکل 9 دیده می شود، Replica 2نیز مقدار s2_eventual=100 را برگردانده است. این یعنی با اینکه Replica 3 خاموش بوده است، پیام replication از Replica 1 به Replica 2 رسیده و Replica 2 نیز به روز شده است. بنابراین در مدلeventual consistency ، سیستم توانست با وجود خرابی یک replica همچنان عملیات PUT را بپذیرد و حداقل replica های روشن را به روز کند. زمان پاسخ client در این خواندن حدود 7.642 میلی ثانیه بود.

سپس تلاش شد مقدار از Replica 3 خوانده شود:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key s2_eventual -mode eventual
2026/06/20 13:23:23 Get "http://localhost:8003/get?key=s2_eventual&mode=eventual": dial tcp [::1]:8003: connectex: No co
nnection could be made because the target machine actively refused it.
exit status 1
```

شکل 10. خاموش بودن replica3 و خطای دسترسی به آن

در شکل 10 مشاهده می شود که اتصال به Replica 3 برقرار نشد و خطای connection refused دریافت شد. این نتیجه طبیعی است، زیرا Replica 3 در ابتدای سناریو متوقف شده بود و دیگر روی پورت 8003 در حال اجرا نبود. بنابراین این replica نمی توانست درخواست GET را پاسخ دهد و همچنین پیام های replicationرا در زمان خاموش بودن دریافت کند. در ادامه، رفتار strong consistency بررسی شد. در این مرحله همچنان Replica 3 خاموش بود، اما Replica 1 و Replica 2 فعال بودند. سپس یک عملیات PUTدر حالت strong روی Replica 1 انجام شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s2_strong -value 200 -mode st
rong
HTTP status: 200 OK
Client measured latency: 4043.415 ms
{
  "ok": true,
  "message": "PUT committed with majority acknowledgement",
  "record": {
    "key": "s2_strong",
    "value": "200",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781949456978171500
  },
  "updated_replicas": 2,
  "latency_ms": 4034.986
}
```

شکل 11. موفق شدن PUT در حالت strong با وجود خاموش بودن یک replica

در شکل 11 مشاهده می شود که عملیات PUT در حالت strong با موفقیت انجام شده است. وضعیت HTTP برابر 200 OK و مقدار ok برابر true است. پیام خروجی نشان می دهد که عملیات با تایید اکثریت انجام شده است. مقدار updated_replicas برابر 2 ثبت شده است. این موضوع نشان می دهد که مقدار جدید حداقل روی دو replica اعمال شده است. چون در این سیستم سه replica وجود دارد، اکثریت برابر با دو replica

است. بنابراین با وجود خاموش بودن Replica 3 ، چون Replica 1 و Replica 2 در دسترس بودند، اکثریت تشکیل شد و عملیات strong موفق شد. زمان پاسخ client برای این عملیات حدود 4043.415 میلی ثانیه بود. دلیل زیاد بودن این زمان، وجود تاخیر مصنوعی replication برابر با حدود 4000 میلی ثانیه در تنظیمات آزمایش بود، زیرا در مدل strong ، client باید منتظر تایید replica دیگر بماند. سپس مقدار s2_strong از Replica 2 در حالت strong خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s2_strong -mode strong
HTTP status: 200 OK
Client measured latency: 15.162 ms
{
  "ok": true,
  "message": "coordinated strong read; newest version returned",
  "record": {
    "key": "s2_strong",
    "value": "200",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781949456978171500
  },
  "checked_replicas": 2,
  "latency_ms": 7.292
}
```

شکل 12. خواندن مقدار strong از Replica 2 پس از تایید اکثریت

همان طور که در شکل 12 دیده می شود، Replica 2مقدار s2_strong=200 را با موفقیت برگردانده است. در خروجی، پیام coordinated strong read; newest version returned نمایش داده شده است. این یعنی خواندن در حالت strong به صورت هماهنگ شده انجام شده و جدیدترین نسخه مقدار برگشت داده شده است. همچنین checked_replicas برابر 2 است، یعنی در این خواندن دو replica بررسی شده اند. زمان پاسخ client برای این GET حدود 15.162 میلی ثانیه ثبت شد. برای بررسی حالت نبود اکثریت، در ادامه Replica 2نیز متوقف شد و فقط Replica 1 روشن باقی ماند. سپس یک عملیات PUT در حالت strong روی Replica 1 انجام شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s2_strong_fail -value 300 -mo
de strong
HTTP status: 503 Service Unavailable
Client measured latency: 4019.684 ms
{
  "ok": false,
  "message": "PUT failed: majority of replicas is not available",
  "record": {
    "key": "s2_strong_fail",
    "value": "300",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781949622749419600
  },
  "updated_replicas": 1,
  "latency_ms": 4011.763
}
```

شکل 13. نوشتن روی replica1 در صورت خاموش بودن دو تای دیگر

در شکل 13 مشاهده می شود که عملیات PUT با خطای HTTP 503 Service Unavailable شکست خورده است. مقدار ok برابر false است و پیام خروجی برابر PUT failed: majority of replicas is not availableاست. مقدار updated_replicas برابر 1 است، یعنی فقط Replica 1 در دسترس بوده و هیچ replicaدیگری مقدار را تایید نکرده است. چون برای سه replica اکثریت برابر با دو replica است، وجود فقط یک replica برای موفق شدن عملیات strong کافی نیست. بنابراین سیستم به درستی عملیات را رد کرده است. زمان پاسخ client برای این عملیات حدود 4019.684 میلی ثانیه بود. برای اطمینان از اینکه مقدار ناموفق واقعا ذخیره نشده است، پس از شکستPUT ، عملیات GET برای همان key انجام شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8001 -key s2_strong_fail -mode strong
HTTP status: 404 Not Found
Client measured latency: 12.077 ms
{
  "ok": false,
  "message": "key not found",
  "record": {
    "key": "",
    "value": "",
    "version": 0,
    "updated_by": "",
    "updated_at": 0
  },
  "checked_replicas": 1,
  "latency_ms": 4.077
}
```

شکل 14. شکست PUT در حالت strong هنگام نبود اکثریت

در شکل 14 مشاهده می شود که پاسخ برابر key not found است و HTTP status برابر 404 Not Found برگشته است. این نتیجه نشان می دهد که وقتی عملیات PUT در حالت strong به دلیل نبود اکثریت شکست می خورد، مقدار مورد نظر commit نمی شود و در سیستم ذخیره نمی گردد. بنابراین این رفتار با تعریف strong consistencyساده شده در این پروژه سازگار است.

تحلیل سناریو

نتایج این سناریو نشان می دهد که مدل eventual consistency و strong consistency در برابر خرابی replicaرفتار متفاوتی دارند. در مدلeventual consistency ، با خاموش بودن3 Replica ، عملیات PUT روی Replica 1 همچنان موفق شد. مقدار ابتدا روی Replica 1 ذخیره شد و سپس به Replica 2 رسید. بنابراین این مدل availability بالاتری دارد، زیرا حتی با خرابی یک replica نیز replica دریافت کننده می تواند درخواست را بپذیرد و پاسخ موفق بدهد. البته Replica 3 تا زمانی که خاموش باشد مقدار جدید را دریافت نمی کند. در مدلstrong consistency ، با خاموش بودن فقط یکreplica ، عملیات PUT همچنان موفق شد، زیرا اکثریت replica ها هنوز در دسترس بودند. در این آزمایش، Replica 1وReplica 2 فعال بودند و

بنابراین اکثریت دو تایی تشکیل شد. اما وقتی Replica 2 نیز خاموش شد و فقط Replica 1 باقی ماند، عملیات strong PUT شکست خورد. دلیل آن این است که سیستم دیگر نمی توانست اکثریت را به دست آورد. این موضوع نشان می دهد که strong consistency سازگاری بیشتری ایجاد می کند، اما برای موفق شدن عملیات نوشتن به اکثریت نیاز دارد و در صورت نبود اکثریت، availabilityکاهش پیدا می کند. بنابراین نتیجه کلی این سناریو این است که eventual consistency در برابر خرابی replica انعطاف پذیرتر است و حتی در زمان خرابی بخشی از سیستم می تواند پاسخ موفق بدهد، اما ممکن است همه replica ها به روز نباشند. در مقابل، strong consistencyفقط زمانی عملیات را موفق اعلام می کند که اکثریت مقدار را تایید کرده باشند. این کار باعث اطمینان بیشتر از سازگاری داده می شود، اما اگر اکثریت replica ها در دسترس نباشند، عملیات نوشتن شکست می خورد.

متریک های ثبت شده در این سناریو

در بخش eventual consistency :

PUT latencyبرای نوشتن s2_eventual روی Replica 1 :حدود 23.080 میلی ثانیه

GET latencyاز Replica 1 : حدود 7.858 میلی ثانیه

GET latencyاز Replica 2 پس از replication : حدود 7.642 میلی ثانیه

تعداد replica های به روز شده در پاسخ اولیه :PUT برابر 1

وضعیت Replica 3 : خاموش و غیرقابل دسترس

نتیجه: عملیات موفق شد و مقدار به replica روشن دیگر رسید

در بخش strong consistency با یک replica خاموش:

PUT latencyبرای نوشتن s2_strong روی Replica 1 : حدود 4043.415 میلی ثانیه

تعداد replica های به روز شده: برابر 2

GET latencyاز Replica 2 : حدود 15.162 میلی ثانیه

تعداد replica های بررسی شده درGET : برابر 2

نتیجه: عملیات موفق شد، زیرا اکثریت وجود داشت

در بخش strong consistency با دو replica خاموش:

PUT latencyبرای نوشتن s2_strong_fail روی Replica 1 : حدود 4019.684 میلی ثانیه

تعداد replica های به روز شده: برابر 1

وضعیت پاسخHTTP 503 Service Unavailable :

نتیجهPUT : شکست به دلیل نبود اکثریت

نتیجه GET پس از PUT ناموفق:key not found

سناریو 3: تعارض همزمان

هدف این سناریو بررسی حالتی است که تقریبا به صورت همزمان، دو مقدار متفاوت برای یک کلید یکسان روی دو replica مختلف نوشته شوند. در چنین حالتی ممکن است هر replica قبل از دریافت پیام replication از replica دیگر، مقدار محلی خودش را ثبت کند. بنابراین برای یکkey ، دو مقدار متفاوت با version یکسان ایجاد می شود. این وضعیت conflict یا تعارض نام دارد و سیستم باید با استفاده از سیاست conflict resolutionمقدار نهایی را مشخص کند. برای اجرای این سناریو، ابتدا هر سه replica اجرا شدند. سپس مقدار تاخیر replication به گونه ای تنظیم شد که پیام های replication با تاخیر ارسال شوند. این کار باعث شد بین زمان نوشتن مقدار روی Replica 1 و رسیدن آن به Replica 2 فاصله ایجاد شود. در این فاصله، مقدار متفاوتی برای همان key روی Replica 2 نوشته شد. به این ترتیب، conflictبه صورت عملی ایجاد شد.

در مرحله اول، مقدار s3_conflict=10 روی Replica 1 و در حالت eventual consistency نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s3_conflict -value 10 -mode e
ventual
HTTP status: 200 OK
Client measured latency: 25.570 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s3_conflict",
    "value": "10",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781950663909819600
  },
  "updated_replicas": 1,
  "latency_ms": 13.901
}
```

شکل 15. نوشتن مقدار s3_conflict=10 روی Replica 1

همان طور که در شکل 15 مشاهده می شود، عملیات PUT روی Replica 1 با موفقیت انجام شد. مقدار key برابر s3_conflict و value برابر 10 است. همچنین version مقدار برابر 1 و updated_by برابر replica1 ثبت شده است. مقدار updated_replicas برابر 1 است، یعنی در لحظه پاسخ دادن به client ، مقدار فقط روی Replica 1 ذخیره شده و replication به replica های دیگر هنوز در پس زمینه در حال انجام بوده است. زمان پاسخ client برای این عملیات حدود 25.570 میلی ثانیه ثبت شد. تقریبا همزمان با عملیات قبلی، مقدار متفاوتی برای همان key روی Replica 2 نوشته شد. در این مرحله مقدار s3_conflict=20 روی Replica 2و در حالت eventual consistency ثبت شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8002 -key s3_conflict -value 20 -mode e
ventual
HTTP status: 200 OK
Client measured latency: 19.993 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s3_conflict",
    "value": "20",
    "version": 1,
    "updated_by": "replica2",
    "updated_at": 1781950664661313900
  },
  "updated_replicas": 1,
  "latency_ms": 12.186
}
```

شکل 16. نوشتن مقدار s3_conflict=20 روی Replica 2 به صورت همزمان

همان طور که در شکل 16 دیده می شود، عملیات PUT روی Replica 2 نیز با موفقیت انجام شد. در این خروجی، keyبرابرs3_conflict ، valueبرابر 20، versionبرابر 1 و updated_by برابر replica2 است. چون این عملیات قبل از دریافت پیام replication از Replica 1 انجام شده است، Replica 2نیز برای این key نسخه 1 را ایجاد کرده است. بنابراین اکنون دو مقدار متفاوت برای یک key با version یکسان وجود دارد:

در Replica 1 :

s3_conflict = 10, version = 1, updated_by = replica1

در Replica 2 :

s3_conflict = 20, version = 1, updated_by = replica2

این دقیقا حالت conflict است، زیرا version ها برابر هستند اما value ها متفاوت اند. زمان پاسخ client برای PUT روی Replica 2 حدود 19.993 میلی ثانیه بود. برای مشاهده وضعیت سیستم قبل از حل کامل تعارض، مقدار key از Replica 1 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8001 -key s3_conflict -mode eventual
HTTP status: 200 OK
Client measured latency: 8.572 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s3_conflict",
    "value": "10",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781950907355447800
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 17. مقدار محلی Replica 1 قبل از حل کامل تعارض

در شکل 17 مشاهده می شود که Replica 1 مقدار محلی خودش را برگردانده است. مقدار برگشتی برابر 10، versionبرابر 1 و updated_by برابر replica1 است. این نشان می دهد که قبل از حل کامل تعارض، Replica 1 هنوز مقدار نوشته شده توسط خودش را نگه داشته است. زمان پاسخ client برای این خواندن حدود 8.572 میلی ثانیه بود. سپس مقدار همان key از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s3_conflict -mode eventual
HTTP status: 200 OK
Client measured latency: 7.305 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s3_conflict",
    "value": "20",
    "version": 1,
    "updated_by": "replica2",
    "updated_at": 1781950907906836000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 18. مقدار محلی Replica 2 قبل از حل کامل تعارض

همان طور که در شکل 18 دیده می شود، Replica 2مقدار محلی خودش یعنی 20 را برگردانده است. در این خروجی نیز version برابر 1 است، اما updated_by برابر replica2 است. این نتیجه نشان می دهد که قبل از تکمیل replication و اجرای کامل سیاست حل تعارض، دو replica مقدارهای متفاوتی برای key یکسان داشته اند. بنابراین تعارض همزمان به درستی در سیستم ایجاد شده است. زمان پاسخ client برای این خواندن حدود 7.305 میلی ثانیه بود. پس از گذشت چند ثانیه و تبادل پیام های replication بین replica ها، دوباره مقدار s3_conflict از Replica 1 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8001 -key s3_conflict -mode eventual
HTTP status: 200 OK
Client measured latency: 7.822 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s3_conflict",
    "value": "20",
    "version": 1,
    "updated_by": "replica2",
    "updated_at": 1781950907906836000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 19. مقدار نهایی Replica 1 پس از حل تعارض

در شکل 19 مشاهده می شود که مقدار نهایی روی Replica 1 برابر 20 شده است. همچنین version برابر 1 و updated_by برابر replica2 است. این یعنی Replica 1 پس از دریافت مقدار Replica 2 و اجرای سیاستconflict resolution ، مقدار محلی قبلی خود یعنی 10 را کنار گذاشته و مقدار 20 را پذیرفته است. زمان پاسخ client برای این خواندن حدود 7.822 میلی ثانیه بود. سپس مقدار نهایی از Replica 2 نیز خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s3_conflict -mode eventual
HTTP status: 200 OK
Client measured latency: 7.818 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s3_conflict",
    "value": "20",
    "version": 1,
    "updated_by": "replica2",
    "updated_at": 1781950907906836000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 20. مقدار نهایی Replica 2 پس از حل تعارض

در شکل 20 دیده می شود که Replica 2 همچنان مقدار 20 را برای key برابر s3_conflict نگه داشته است. مقدار updated_by نیز برابر replica2 است. این نتیجه طبیعی است، زیرا طبق سیاست حل تعارض، مقدار Replica 2برنده تعارض بوده است. زمان پاسخ client برای این خواندن حدود 7.818 میلی ثانیه بود. در نهایت، برای بررسی همگرایی کامل سیستم، مقدار از Replica 3 نیز خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key s3_conflict -mode eventual
HTTP status: 200 OK
Client measured latency: 7.986 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s3_conflict",
    "value": "20",
    "version": 1,
    "updated_by": "replica2",
    "updated_at": 1781950907906836000
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 21. مقدار نهایی Replica 3 پس از حل تعارض

همان طور که در شکل 21 مشاهده می شود، Replica 3نیز مقدار s3_conflict=20 را برگردانده است. مقدار versionبرابر 1 و updated_by برابر replica2 است. این نشان می دهد که پس از تکمیل replication و حل تعارض، هر سه replica به مقدار یکسان رسیده اند. بنابراین سیستم در نهایت همگرا شده است. زمان پاسخ client برای این خواندن حدود 7.986 میلی ثانیه بود.

تحلیل سناریو

این سناریو نشان می دهد که در یک سیستمreplicated ، اگر دو عملیات نوشتن تقریبا همزمان روی دو replicaمختلف انجام شوند، ممکن است conflict ایجاد شود. در این آزمایش، Replica 1مقدار s3_conflict=10را با version برابر 1 تولید کرد و تقریبا همزمان Replica 2 مقدار s3_conflict=20 را نیز با version برابر 1 تولید کرد. چون هر دو مقدار version یکسان داشتند ولی value آن ها متفاوت بود، سیستم با یک تعارض واقعی مواجه شد.

برای حل این تعارض، سیاست conflict resolution پیاده سازی شده در سیستم اجرا شد. در این سیاست، ابتدا version مقایسه می شود. اگر version یک مقدار بیشتر باشد، همان مقدار جدیدتر انتخاب می شود. اما در این سناریو، versionهر دو مقدار برابر 1 بود. بنابراین معیار دوم یعنی شناسه replica به روز کننده استفاده شد. طبق سیاست انتخاب شده، اگر version ها برابر باشند، مقدار مربوط به replica با شناسه بزرگ تر انتخاب می شود. چون replica2 از replica1 بزرگ تر در نظر گرفته می شود، مقدار نوشته شده توسط Replica 2 یعنی 20 به عنوان مقدار نهایی انتخاب شد.

نتیجه نهایی نیز این تحلیل را تایید می کند. قبل از حل کامل تعارض، Replica 1مقدار 10 و Replica 2 مقدار 20 را نشان می دادند. اما پس از گذشت چند ثانیه و تبادل پیام هایreplication ، هر سه replica

مقدار 20 را برای key برابر s3_conflict نشان دادند. بنابراین سیستم توانست تعارض را به صورت قطعی حل کند و replica ها در نهایت به یک مقدار مشترک رسیدند.

متریک های ثبت شده در این سناریو

PUT latencyبرای نوشتن مقدار 10 روی1 Replica : حدود 25.570 میلی ثانیه

PUT latencyبرای نوشتن مقدار 20 روی2 Replica : حدود 19.993 میلی ثانیه

GET latencyاز 1 Replica قبل از حل کامل تعارض: حدود 8.572 میلی ثانیه

GET latencyاز 2 Replica قبل از حل کامل تعارض: حدود 7.305 میلی ثانیه

GET latencyاز 1 Replica پس از حل تعارض: حدود 7.822 میلی ثانیه

GET latencyاز 2 Replica پس از حل تعارض: حدود 7.818 میلی ثانیه

GET latencyاز 3 Replica پس از حل تعارض: حدود 7.986 میلی ثانیه

تعداد replica های به روز شده در پاسخ اولیه هرPUT : برابر 1

مقدار اولیه 1 Replica قبل از حل تعارض: 10

مقدار اولیه 2 Replica قبل از حل تعارض: 20

مقدار نهایی هر سه replica پس از حل تعارض: 20

سیاست حل تعارض: در صورت برابر بودنversion ، مقدار مربوط به replica با شناسه بزرگ تر انتخاب می شود

نتیجه نهایی: مقدار نوشته شده توسط 2 Replica برنده شد و همه replica ها به مقدار 20 همگرا شدند

سناریو 4: تاثیر تاخیر شبکه

هدف این سناریو بررسی تاثیر تاخیر شبکه بر رفتار سیستم در دو مدل eventual consistency و strong consistency است. در این آزمایش، تاخیر مصنوعی replication در سه حالت مختلف برابر با 0ms، 500 ms و 2000 ms تنظیم شد. سپس در هر حالت، عملیات نوشتن و خواندن در مدل eventual consistency و strong consistency اجرا شد تا اثر تاخیر روی زمان پاسخ، زمان همگرایی replica ها و احتمال رخ دادن stale read بررسی شود. در این سناریو، همه replica ها فعال بودند و فقط مقدار replication_delay_ms در فایل های config تغییر داده شد. پس از هر تغییر delay ، replicaها دوباره اجرا شدند و داده های قبلی پاک شدند تا نتیجه آزمایش ها با یکدیگر تداخل نداشته باشند.

بخش اول: تاخیر شبکه برابر با 0ms

در مرحله اول، مقدار replication_delay_ms در هر سه فایل config برابر با 0 قرار داده شد. سپس هر سه replica دوباره اجرا شدند. هدف این بخش این بود که رفتار سیستم در حالتی بررسی شود که عملا تاخیر مصنوعی برای replication وجود ندارد.

```
PS D:\Dist\Projects\HW3> go run replica/main.go -config configs/replica1.json
2026/06/20 14:18:56 starting with empty data store: open results/replica1_data.json: The system cannot find the file spe
cified.
2026/06/20 14:18:56 replica1 is running on :8001
2026/06/20 14:18:56 peers: [http://localhost:8002 http://localhost:8003]
2026/06/20 14:18:56 replication delay: 0 ms
```

شکل 22. اجرا روی هر سه با تاخیر 0 ms

همان طور که در شکل 22 مشاهده می شود، Replica 1 روی پورت 8001 اجرا شده و مقدار replication delay برابر با 0 ms است. همین تنظیم برای Replica 2 و Replica 3 نیز اعمال شده بود. ابتدا مدل eventual consistency بررسی شد. در این حالت، مقدار s4_e_0=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_e_0 -value 100 -mode event
ual
HTTP status: 200 OK
Client measured latency: 24.230 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s4_e_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781952709075400100
  },
  "updated_replicas": 1,
  "latency_ms": 11.479
}
```

شکل 23. نوشتن مقدار در حالت eventual با تاخیر 0ms

در شکل 23 مشاهده می شود که عملیات PUT با موفقیت انجام شده است. مقدار key برابر s4_e_0 و value برابر 100 است. همچنین version برابر 1 و updated_by برابر replica1 ثبت شده است. مقدار updated_replicas برابر 1 است، زیرا در مدل eventual consistency پاسخ client پس از اعمال مقدار روی replica محلی برگردانده می شود و replication به صورت غیرهمزمان انجام می گیرد. زمان پاسخ clientبرای این PUT حدود 24.230 میلی ثانیه و latency داخلی گزارش شده حدود 11.479 میلی ثانیه بود. سپس بلافاصله مقدار s4_e_0 از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_0 -mode eventual
HTTP status: 200 OK
Client measured latency: 8.413 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781952709075400100
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 24. خواندن فوری از Replica 2 در حالت eventual با تاخیر 0ms

در شکل 24 مشاهده می شود که Replica 2 مقدار جدید را با موفقیت برگردانده است. این یعنی با تاخیر 0ms، پیام replication بسیار سریع به Replica 2 رسیده و در خواندن فوری stale read مشاهده نشده است. زمان پاسخ client برای این GET حدود 8.413 میلی ثانیه بود. برای اطمینان از همگرایی، دوباره از Replica 2 و سپس از Replica 3 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_0 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.786 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781952709075400100
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 25. مقدار Replica 2 پس از همگرایی با تاخیر 0ms

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key s4_e_0 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.942 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781952709075400100
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 26. مقدار Replica 3 پس از همگرایی با تاخیر 0ms

در شکل های 25 و 26 مشاهده می شود که هر دو replica مقدار s4_e_0=100 را برگردانده اند. بنابراین در حالت delay برابر 0ms، سیستم تقریبا بلافاصله همگرا شده است. زمان پاسخ GET از Replica 2 حدود 7.786 میلی ثانیه و از Replica 3 حدود 7.942 میلی ثانیه بود. سپس مدل strong consistency با همان تاخیر 0 msبررسی شد. مقدار s4_s_0=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_s_0 -value 100 -mode stron
g
HTTP status: 200 OK
Client measured latency: 33.476 ms
{
  "ok": true,
  "message": "PUT committed with majority acknowledgement",
  "record": {
    "key": "s4_s_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953041085019500
  },
  "updated_replicas": 3,
  "latency_ms": 26.045
}
```

شکل 27. نوشتن مقدار در حالت strong با تاخیر 0ms

در شکل 27 دیده می شود که عملیات strong PUT با موفقیت انجام شده است. مقدار updated_replicas برابر 3 است، یعنی هر سه replica مقدار جدید را تایید کرده اند. زمان پاسخ client حدود 33.476 میلی ثانیه و latency داخلی حدود 26.045 میلی ثانیه بود. چون تاخیر replication برابر 0 msبود، strong PUT نیز با latency کمی انجام شد. سپس مقدار s4_s_0 از Replica 2 در حالت strong خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_s_0 -mode strong
HTTP status: 200 OK
Client measured latency: 38.045 ms
{
  "ok": true,
  "message": "coordinated strong read; newest version returned",
  "record": {
    "key": "s4_s_0",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953041085019500
  },
  "checked_replicas": 3,
  "latency_ms": 19.274
}
```

شکل 28. خواندن مقدار از Replica 2 در حالت strong با تاخیر 0ms

در شکل 28 مشاهده می شود که مقدار جدید با موفقیت برگشته است. پیام خروجی نشان می دهد که خواندن به صورت هماهنگ شده انجام شده و جدیدترین نسخه برگشته است. مقدار checked_replicas برابر 3 است، یعنی سه replica بررسی شده اند. زمان پاسخ client حدود 38.045 میلی ثانیه و latency داخلی حدود 19.274 میلی ثانیه بود.

بخش دوم: تاخیر شبکه برابر با ms500

در مرحله دوم، مقدار replication_delay_ms در فایل های config برابر با 500 قرار داده شد و replica ها دوباره اجرا شدند.

```
PS D:\Dist\Projects\HW3> go run replica/main.go -config configs/replica1.json
2026/06/20 14:30:28 starting with empty data store: open results/replica1_data.json: The system cannot find the file spe
cified.
2026/06/20 14:30:28 replica1 is running on :8001
2026/06/20 14:30:28 peers: [http://localhost:8002 http://localhost:8003]
2026/06/20 14:30:28 replication delay: 500 ms
```

شکل 29. اجرا روی هر سه با تاخیر 500 ms

در شکل 29 مشاهده می شود که مقدار replication delay برابر با 500 msتنظیم شده است. در این حالت انتظار می رود replication نسبت به حالت قبل کندتر انجام شود و احتمال مشاهده ناسازگاری موقت در مدل eventual consistencyبیشتر شود. ابتدا در حالت eventual consistency مقدار s4_e_500=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_e_500 -value 100 -mode eve
ntual
HTTP status: 200 OK
Client measured latency: 26.723 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s4_e_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953398163068500
  },
  "updated_replicas": 1,
  "latency_ms": 14.421
}
```

شکل 30. نوشتن مقدار در حالت eventual با تاخیر 500ms

در شکل 30 مشاهده می شود که عملیات PUT موفق بوده است. مقدار updated_replicas برابر 1 است، یعنی در زمان پاسخ دادن به client مقدار فقط روی Replica 1 ذخیره شده بود. زمان پاسخ client حدود 26.723 میلی ثانیه و latency داخلی حدود 14.421 میلی ثانیه بود. این نتیجه نشان می دهد که در مدل eventual consistency، حتی با افزایشdelay ، زمان پاسخ PUT همچنان کم باقی می ماند، چون client منتظر تکمیل replication نمی ماند. سپس مقدار از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_500 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.720 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953398163068500
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 31. خواندن فوری از Replica 2 در حالت eventual با تاخیر 500ms

در شکل 31 دیده می شود که Replica 2 مقدار جدید را برگردانده است. در این آزمایش، stale readدر حالت 500 msمشاهده نشد. دلیل این موضوع می تواند این باشد که فاصله زمانی بین اجرای PUT و GET کافی بوده یا replication قبل از اجرای GET کامل شده است. با این حال، چون مدل eventual consistency غیرهمزمان است، در صورت اجرای سریع تر GET همچنان امکان مشاهده مقدار قدیمی یا key not found وجود دارد. زمان پاسخ client در این GET حدود 7.720 میلی ثانیه بود. برای بررسی همگرایی، دوباره مقدار از Replica 2 و سپس از Replica 3 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_500 -mode eventual
HTTP status: 200 OK
Client measured latency: 8.042 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953398163068500
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 32. مقدار Replica 2 پس از همگرایی با تاخیر 500ms

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key s4_e_500 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.892 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953398163068500
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 33. مقدار Replica3 پس از همگرایی با تاخیر 500ms

در شکل های 32 و 33 مشاهده می شود که هر دو replica مقدار s4_e_500=100 را نشان می دهند. بنابراین بعد از تکمیلreplication ، هر سه replica همگرا شده اند. زمان پاسخ از Replica 2 حدود 8.042 میلی ثانیه و از Replica 3 حدود 7.892 میلی ثانیه بود.

سپس strong consistency با تاخیر 500 msبررسی شد. مقدار s4_s_500=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_s_500 -value 100 -mode str
ong
HTTP status: 200 OK
Client measured latency: 542.737 ms
{
  "ok": true,
  "message": "PUT committed with majority acknowledgement",
  "record": {
    "key": "s4_s_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953593855669600
  },
  "updated_replicas": 3,
  "latency_ms": 533.741
}
```

شکل 34. نوشتن مقدار در حالت strong با تاخیر 500ms

در شکل 34 مشاهده می شود که عملیات strong PUT موفق بوده است و مقدار updated_replicas برابر 3 است. زمان پاسخ client حدود 542.737 میلی ثانیه و latency داخلی حدود 533.741 میلی ثانیه بود. این افزایش latency نسبت به حالت 0 msکاملا قابل انتظار است، زیرا درstrong consistency ، عملیات PUTباید منتظر تایید replica های دیگر بماند و تاخیر شبکه مستقیما روی زمان پاسخ اثر می گذارد. سپس مقدار s4_s_500 از Replica 2 در حالت strong خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_s_500 -mode strong
HTTP status: 200 OK
Client measured latency: 42.323 ms
{
  "ok": true,
  "message": "coordinated strong read; newest version returned",
  "record": {
    "key": "s4_s_500",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953593855669600
  },
  "checked_replicas": 3,
  "latency_ms": 20.872
}
```

شکل 35. خواندن مقدار از Replica 2 در حالت strong با تاخیر 500ms

در شکل 35 مشاهده می شود که مقدار جدید با موفقیت برگشته است. مقدار checked_replicas برابر 3 است و پیام خروجی نشان می دهد که جدیدترین نسخه برگشت داده شده است. زمان پاسخ client حدود 42.323 میلی ثانیه و latency داخلی حدود 20.872 میلی ثانیه بود.

بخش سوم: تاخیر شبکه برابر با ms2000

در مرحله سوم، مقدار replication_delay_ms در فایل های config برابر با 2000 قرار داده شد و replica ها دوباره اجرا شدند.

```
PS D:\Dist\Projects\HW3> go run replica/main.go -config configs/replica1.json
2026/06/20 14:38:45 starting with empty data store: open results/replica1_data.json: The system cannot find the file spe
cified.
2026/06/20 14:38:45 replica1 is running on :8001
2026/06/20 14:38:45 peers: [http://localhost:8002 http://localhost:8003]
2026/06/20 14:38:45 replication delay: 2000 ms
```

شکل 36. اجرا روی هر سه با تاخیر 500 ms

در شکل 36 مشاهده می شود که مقدار replication delay برابر با 2000 msاست. این بخش مهم ترین بخش سناریو است، زیرا با افزایش تاخیر، اثر آن روی stale read و زمان همگرایی واضح تر دیده می شود.

ابتدا در حالت eventual consistency مقدار s4_e_2000=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_e_2000 -value 100 -mode ev
entual
HTTP status: 200 OK
Client measured latency: 23.782 ms
{
  "ok": true,
  "message": "PUT applied locally; replication is running asynchronously",
  "record": {
    "key": "s4_e_2000",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953827179352500
  },
  "updated_replicas": 1,
  "latency_ms": 13.05
}
```

شکل 37. نوشتن مقدار در حالت eventual با تاخیر ms2000

در شکل 37 مشاهده می شود که عملیات PUT با موفقیت انجام شده است. مقدار updated_replicas برابر 1 است، یعنی در لحظه پاسخ به client مقدار فقط روی Replica 1 ذخیره شده بود. زمان پاسخ client حدود 23.782 میلی ثانیه و latency داخلی حدود 13.05 میلی ثانیه بود. این نشان می دهد که حتی با افزایش تاخیر به ms2000، زمان پاسخ PUT در eventual consistency همچنان کم باقی مانده است، چون replicationبه صورت غیرهمزمان انجام می شود. سپس بلافاصله مقدار از Replica 2 خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_2000 -mode eventual
HTTP status: 404 Not Found
Client measured latency: 8.011 ms
{
  "ok": false,
  "message": "key not found",
  "record": {
    "key": "",
    "value": "",
    "version": 0,
    "updated_by": "",
    "updated_at": 0
  },
  "checked_replicas": 0,
  "latency_ms": 0
}
```

شکل 38. خواندن فوری از Replica 2 در حالت eventual با تاخیر ms2000

در شکل 38 مشاهده می شود که پاسخ برابر key not found است و وضعیت HTTP برابر 404 Not Found برگشته است. این نتیجه نشان می دهد که Replica 2 هنوز پیام replication مربوط به s4_e_2000=100 را دریافت نکرده بود. بنابراین در این مرحله یک stale read یا ناسازگاری موقت مشاهده شد. زمان پاسخ client برای این GET حدود 8.011 میلی ثانیه بود.پس از گذشت زمان کافی برای تکمیلreplication ، مقدار از Replica 2دوباره خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_e_2000 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.538 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_2000",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953827179352500
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 39. مقدار Replica 2 پس از همگرایی با تاخیر 2000ms

در شکل 39 مشاهده می شود که Replica 2 این بار مقدار s4_e_2000=100 را با موفقیت برگردانده است. این یعنی پیام replication پس از تاخیر شبکه به Replica 2 رسیده و replica به روز شده است. زمان پاسخ clientدر این خواندن حدود 7.538 میلی ثانیه بود.

برای اطمینان از همگرایی کامل، مقدار از Replica 3 نیز خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8003 -key s4_e_2000 -mode eventual
HTTP status: 200 OK
Client measured latency: 7.992 ms
{
  "ok": true,
  "message": "local read from this replica",
  "record": {
    "key": "s4_e_2000",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781953827179352500
  },
  "checked_replicas": 1,
  "latency_ms": 0
}
```

شکل 40. مقدار Replica 3 پس از همگرایی با تاخیر 2000ms

در شکل 40 مشاهده می شود که Replica 3 نیز مقدار s4_e_2000=100 را برگردانده است. بنابراین پس از گذشت زمان کافی، هر سه replica به مقدار یکسان رسیده اند. زمان پاسخ client از Replica 3 حدود 7.992 میلی ثانیه بود .در ادامه، strong consistencyبا تاخیر 2000 msبررسی شد. مقدار s4_s_2000=100 روی Replica 1 نوشته شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op put -replica http://localhost:8001 -key s4_s_2000 -value 100 -mode st
rong
HTTP status: 200 OK
Client measured latency: 2041.933 ms
{
  "ok": true,
  "message": "PUT committed with majority acknowledgement",
  "record": {
    "key": "s4_s_2000",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781954028522748900
  },
  "updated_replicas": 3,
  "latency_ms": 2033.365
}
```

شکل 41. نوشتن مقدار در حالت strong با تاخیر 2000ms

در شکل 41 دیده می شود که عملیات strong PUT موفق بوده است و مقدار updated_replicas برابر 3 است. زمان پاسخ client حدود 2041.933 میلی ثانیه و latency داخلی حدود 2033.365 میلی ثانیه ثبت شده است. این نتیجه نشان می دهد که در مدل strong consistency ، تاخیر شبکه تقریبا مستقیما به زمان پاسخ PUT اضافه می شود، زیرا client باید منتظر تایید replica های دیگر بماند.

سپس مقدار s4_s_2000 از Replica 2 در حالت strong خوانده شد:

```
PS D:\Dist\Projects\HW3> go run client/main.go -op get -replica http://localhost:8002 -key s4_s_2000 -mode strong
HTTP status: 200 OK
Client measured latency: 10.573 ms
{
  "ok": true,
  "message": "coordinated strong read; newest version returned",
  "record": {
    "key": "s4_s_2000",
    "value": "100",
    "version": 1,
    "updated_by": "replica1",
    "updated_at": 1781954028522748900
  },
  "checked_replicas": 3,
  "latency_ms": 1.593
}
```

شکل 42. خواندن مقدار از Replica 2 در حالت strong با تاخیر 2000ms

در شکل 42 مشاهده می شود که مقدار جدید با موفقیت برگشته است. مقدار checked_replicas برابر 3 است و پیام خروجی نشان می دهد که جدیدترین نسخه برگشت داده شده است. زمان پاسخ client برای این GETحدود 10.573 میلی ثانیه و latency داخلی حدود 1.593 میلی ثانیه بود.

تحلیل سناریو

نتایج این سناریو نشان می دهد که تاخیر شبکه اثر متفاوتی روی eventual consistency و strong consistencyدارد. در مدل eventual consistency ، عملیات PUT ابتدا فقط روی replica دریافت کننده

اعمال می شود و سپس replication به صورت غیرهمزمان انجام می شود. به همین دلیل، زمان پاسخ PUT در هر سه حالت delay تقریبا کم باقی ماند. برای مثال، PUTدر حالت eventual با تاخیر 0 msحدود 24.230 میلی ثانیه، با تاخیر 500 msحدود 26.723 میلی ثانیه و با تاخیر 2000 msحدود 23.782 میلی ثانیه طول کشید. بنابراین افزایش delay تاثیر زیادی روی latency اولیه PUT در eventual consistency نداشت.

اما اثر delay در مدل eventual consistency بیشتر روی زمان همگرایی و احتمال stale read دیده شد. در تاخیر ms0، مقدار جدید در خواندن فوری از Replica 2 دیده شد و stale read رخ نداد. در تاخیر 500 ms نیز در این اجرای خاص stale read مشاهده نشد، چون احتمالا replication قبل از GET کامل شده بود. اما در تاخیر ms2000، خواندن فوری از Replica 2 باعث برگشت key not found شد. این نشان می دهد که با افزایش تاخیر شبکه، بازه زمانی ناسازگاری بین replica ها بیشتر می شود و احتمال مشاهده stale read افزایش پیدا می کند.

در مقابل، در مدلstrong consistency ، عملیات PUT فقط زمانی موفق اعلام می شود که replica های دیگر مقدار را تایید کنند. بنابراین افزایش delay مستقیما باعث افزایش زمان پاسخ PUT شد. در این آزمایش، strong PUTبا تاخیر 0 msحدود 33.476 میلی ثانیه طول کشید، با تاخیر 500 msبه حدود 542.737 میلی ثانیه رسید و با تاخیر 2000 msحدود 2041.933 میلی ثانیه زمان برد. این افزایش نشان می دهد که strong consistencyسازگاری بیشتری فراهم می کند، اما هزینه آن افزایش latency است.

از نظر تعداد replica های به روز شده نیز دریeventual consistency ، مقدار updated_replicas در پاسخ اولیه PUT برابر 1 بود، زیرا پاسخ قبل از تکمیل replication برگردانده می شود. اما بعد از گذشت زمان کافی، هر سه replica مقدار جدید را داشتند. درstrong consistency ، مقدار updated_replicas در هر سه آزمایش برابر 3 بود، چون هر سه replica در دسترس بودند و عملیات پس از تایید همه آن ها موفق شد.

متریک های ثبت شده در این سناریو

تاخیر ms0، مدل eventual :

PUT latency: حدود 24.230 میلی ثانیه

GETفوری از Replica 2 : حدود 8.413 میلی ثانیه

نتیجه GET فوری: مقدار جدید دیده شد

Stale read: مشاهده نشد

زمان همگرایی: تقریبا فوری

Updated replicas در پاسخ اولیه PUT : برابر 1

تاخیر ms0، مدل strong :

PUT latency: حدود 33.476 میلی ثانیه

GET از Replica 2 : حدود 38.045 میلی ثانیه

Updated replicas: برابر 3

Checked replicas در GET : برابر 3

نتیجه: عملیات با latency کم و با تایید همه replica ها موفق شد

تاخیر ms500، مدل eventual :

PUT latency: حدود 26.723 میلی ثانیه

GET فوری از Replica 2 : حدود 7.720 میلی ثانیه

نتیجه GET فوری: مقدار جدید دیده شد

Stale read: در این اجرا مشاهده نشد

زمان همگرایی: کمتر از حدود 1 ثانیه

Updated replicas در پاسخ اولیه PUT : برابر 1

تاخیر ms500، مدل strong :

PUT latency: حدود 542.737 میلی ثانیه

GET از Replica 2 : حدود 42.323 میلی ثانیه

Updated replicas:برابر 3

Checked replicasدرGET : برابر 3

نتیجه: عملیات موفق شد، اما latency نسبت به 0 msافزایش یافت

تاخیر ms2000، مدلeventual :

PUT latency: حدود 23.782 میلی ثانیه

GETفوری از2 Replica : حدود 8.011 میلی ثانیه

نتیجه GET فوری:key not found

Stale read: یک مورد مشاهده شد

زمان همگرایی: حدود 2 تا 3 ثانیه

Updated replicasدر پاسخ اولیهPUT : برابر 1

تاخیر ms2000، مدلstrong :

PUT latency:حدود 2041.933 میلی ثانیه

GETاز2 Replica : حدود 10.573 میلی ثانیه

Updated replicas: برابر 3

Checked replicasدرGET : برابر 3

نتیجه: عملیات موفق شد، اما latency نوشتن تقریبا به اندازه تاخیر شبکه افزایش یافت

جدول متریک ها و تحلیل کلی نتایج

برای مقایسه رفتار سیستم در سناریوهای مختلف، متریک های اصلی شامل Put Latency ، Convergence Time و Stale Reads ثبت شدند. Put Latency زمان لازم برای انجام عملیات PUT را نشان می دهد . Convergence Time مدت زمانی است که طول می کشد تا replica ها پس از یک به روز رسانی به وضعیت یکسان برسند .Stale Reads نیز تعداد دفعاتی را نشان می دهد که در یک عملیات خواندن، مقدار قدیمی یا مقدار پیدا نشده مشاهده شده است.

نتایج جدول نشان می دهد که در سناریوی 1 که مربوط به مشاهده ناسازگاری موقت در مدل eventual consistency بود، یک stale read مشاهده شد. دلیل این موضوع آن بود که پس از نوشتن مقدار روی Replica 1، بلافاصله از Replica 2 خوانده شد و Replica 2 هنوز پیام replication را دریافت نکرده بود. با این حال، پس از حدود 4 ثانیه، replicaها همگرا شدند و مقدار جدید روی Replica 2 و Replica 3 نیز مشاهده شد.

در سناریوی 2، رفتار سیستم در حضور خرابی یک replica بررسی شد. در مدل eventual consistency ، با وجود خاموش بودن Replica 3 ، عملیات PUT با موفقیت انجام شد و مقدار به Replica 2 رسید. در این حالت stale readمشاهده نشد، اما همگرایی فقط بین replica های روشن انجام شد. در مدل strong consistency نیز با وجود خاموش بودن یک replica ، عملیات PUT موفق شد، زیرا اکثریت هنوز در دسترس بود. با این حال، latencyعملیات PUT به حدود 4 ثانیه رسید، زیرا سیستم برای تایید اکثریت منتظر ماند.

در سناریوی 3، تعارض همزمان بررسی شد. در این سناریو دو عملیات PUT تقریبا همزمان روی Replica 1 و Replica 2انجام شدند و دو مقدار متفاوت برای یک key یکسان تولید شد. در این حالت تمرکز اصلی روی conflict resolutionبود، نه .stale read پس از حدود 4 تا 5 ثانیه، سیستم با استفاده از سیاست حل تعارض به یک مقدار نهایی مشترک همگرا شد.

در سناریوی 4، تاثیر تاخیر شبکه به صورت دقیق تر بررسی شد. نتایج نشان می دهد که در مدل eventual consistency، افزایش تاخیر شبکه تاثیر زیادی روی Put Latency نداشت و زمان پاسخ PUT تقریبا در هر سه حالت 0ms، 500 msو 2000 msکم باقی ماند. اما با افزایشdelay ، زمان همگرایی بیشتر شد و احتمال stale readنیز افزایش پیدا کرد. در delay برابر 2000ms، یک stale read مشاهده شد، زیرا خواندن فوری

از Replica 2 با پاسخ key not found مواجه شد. در مقابل، در مدل strong consistency ، تاخیر شبکه مستقیما روی Put Latency اثر گذاشت. در delay برابر ms0، زمان PUT حدود 33 میلی ثانیه بود، اما با delayبرابر ms 500به حدود 543 میلی ثانیه و با delay برابر ms 2000به حدود 2042 میلی ثانیه رسید. این نتیجه نشان می دهد که strong consistency سازگاری بیشتری فراهم می کند، اما هزینه آن افزایش محسوس latency در زمان افزایش تاخیر شبکه است. به طور کلی، داده های جدول به خوبی trade-off بین consistencyو performance را نشان می دهند. دریeventual consistency ، عملیات نوشتن سریع تر انجام می شود، اما ممکن است تا مدت کوتاهی ناسازگاری و stale read رخ دهد. درstrong consistency ، stale readکمتر است و داده ها هماهنگ تر باقی می مانند، اما زمان پاسخ PUT به شدت به تاخیر شبکه وابسته است. بنابراین این جدول به صورت خلاصه نشان می دهد که هر مدل consistency چه مزایا و چه هزینه هایی دارد.

Stale Reads	Put Latency	Convergence Time	Model	Scenario
1	28.106 ms	حدود 4 ثانیه	Eventual	1
0	23.080 ms	حدود 4 ثانیه برای Replica 2 | Replica 3خاموش بود	Eventual	2
0	4043.415 ms	حدود 4 ثانیه و همزمان با تایید اکثریت	Strong	2
0	25.570 و ms 19.993	حدود 4 تا 5 ثانیه	Eventual	3
0	24.230 ms	تقریبا فوری	Eventual	4(0ms)
0	33.476 ms	تقریبا فوری و همزمان باcommit	Strong	4(0ms)
0	26.723 ms	کمتر از 1 ثانیه	Eventual	4(500ms)
0	542.737 ms	حدود 0.5 ثانیه و همزمان با commit	Strong	4(500ms)
1	23.782 ms	حدود 2 تا 3 ثانیه	Eventual	4(2000ms)
0	2041.933 ms	حدود 2 ثانیه و همزمان با commit	Strong	4(2000ms)